RESEARCH REPORT
DECODER-ONLY TRANSFORMER

Naman Goel

0.1 DATA PREPROCESSING, PACKING, AND CONDITIONING ENVELOPES

0.1.1 Intuition and Theoretical Framework

To build an autoregressive foundation model capable of steered inference, raw text data cannot simply be appended together. The model must learn the implicit boundaries of structural documents, metadata, and contextual formatting. This is achieved by wrapping raw text sequences within explicit special structural control envelopes (or tokens). These envelopes establish strong, high-probability contextual anchors that the self-attention mechanism uses to separate administrative control states from raw text content.

0.1.2 Implementation Logistics and Code Architecture

For the rap dataset domain adaptation, the data loader parses raw JSON objects and maps them into a structured string using specific operational tags. The data envelope was standardized as follows:

```
<|startoftext|>
<|type|> rap
[genre: boom_bap]
[mood: emotional]
<|content|>
{lyrics_content}
<|endoftext|>
```

This multi-tiered prompting layout guarantees that during generation, initializing the context window with the control tokens narrows the probability distribution to the specialized domain sub-spaces optimized during training.

0.1.3 The Fatal "String-Shredding" Failure Mode

During initial execution, a catastrophic bug was identified in how datasets were split for training.

The Broken Sequence:

```
full_rap_corpus = "".join(conditioned_texts)
train_text, val_text = train_test_split(full_rap_corpus,
   test_size=0.1, random_state=42)
train_string = "\n".join(train_text)
```

Root Cause Analysis:

`train_test_split` expects an iterable array-like structure (a list of documents). By joining the list into a single massive string `full_rap_corpus` before passing it to the split function, python treated the giant string as an iterable list of *individual characters*.

The split function shredded the lyrics character-by-character, scrambling sequences and feeding isolated letters separated by artificial newlines to the training hub. This forced the model to exclusively learn vertical alignment and single-letter token steps.

Engineering Resolution:

The data processing pipeline was modified to enforce array-level grouping during the split, ensuring whole records remain intact before any line-by-line tokenization occurs:

```
train_text, val_text = train_test_split(conditioned_texts,
   test_size=0.1, random_state=42)
```

0.2 BYTE-PAIR ENCODING (BPE) TOKENIZATION FRAMEWORK

0.2.1 Intuition and Mathematical Mechanics

Character-level tokenization features an extremely small vocabulary size, but produces long token sequences that exhaust the attention window's $O(N^2)$ memory footprint. Conversely, word-level tokenization causes an unmanageable vocabulary size that triggers sparse gradients and massive parameter inflation in the final linear projection layer.

Byte-Pair Encoding (BPE) resolves this by dynamically balancing sequence compression ratios against vocabulary scale. Starting with base bytes (IDs 0–127 for standard ASCII), the BPE framework scans the text corpus recursively, finds the most frequently co-occurring byte pairs, and assigns them a new, consolidated token ID.

0.2.2 Implementation Specs

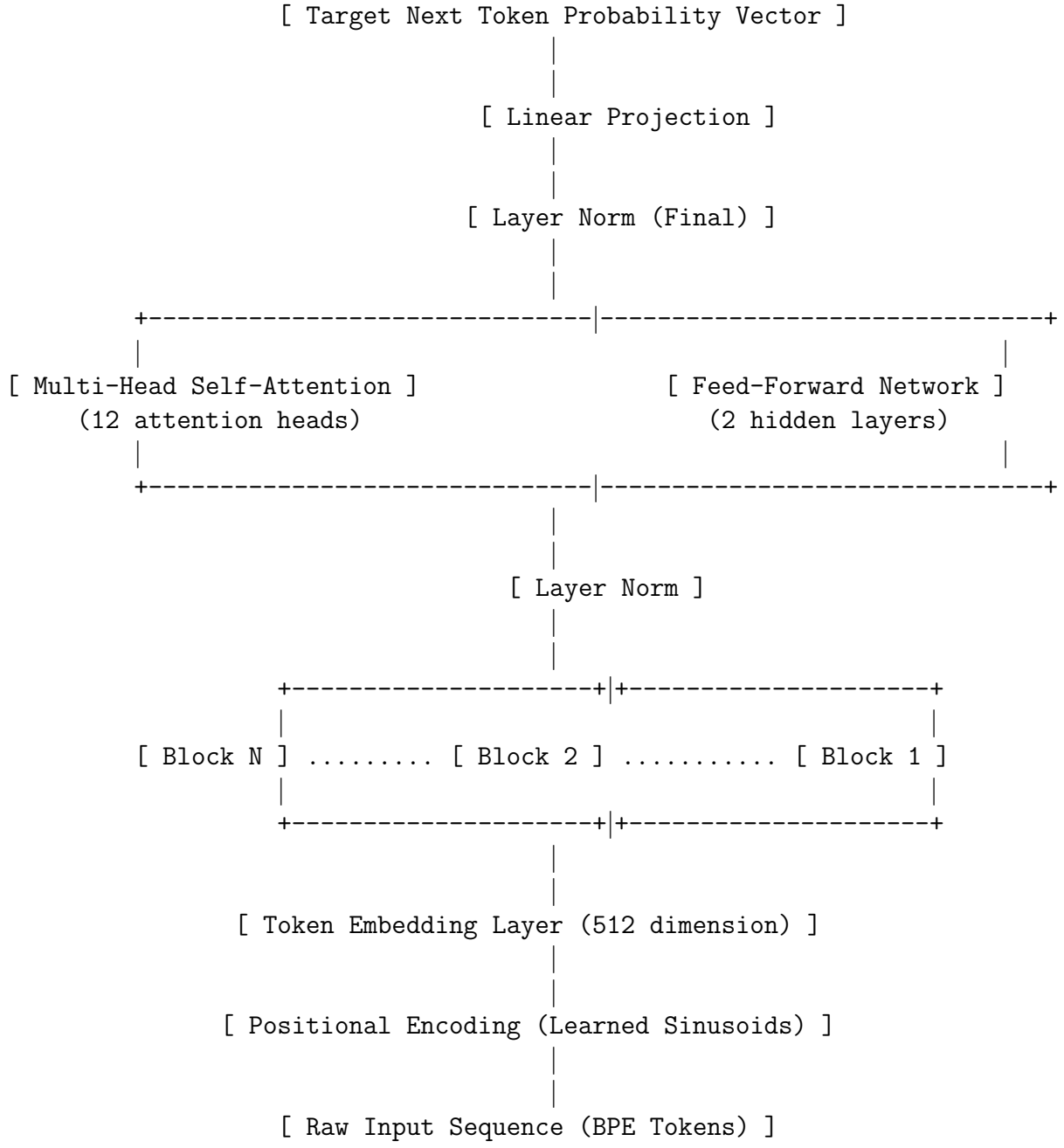
- **Target Vocabulary Size (V):** 512 tokens.
- **Base Vocabulary (0–127):** Standard ASCII character space.
- **Learned Merge Space (128–511):** Statistical frequent character pairs and complete word roots (e.g., `th`, `in`, `the`, `and`, `\n<|`).
- **Metadata Serializer:** Saved to `tokenizer/bpe_meta.pkl` for cross-pipeline deterministic mapping between training and inference scripts.

0.3 GPT DECODER-ONLY ARCHITECTURE

0.3.1 Intuition and Structural Framework

The model utilizes a pure causal, decoder-only autoregressive transformer architecture based on the GPT design principles. Information flows strictly up through parallelized residual streams, managed via Layer Normalization and Multi-Head Causal Self-Attention.

0.3.2 Architectural Blueprint & Layer Configuration



Layer Specification Details:

- **Embedding Dimension (d_{model}):** 512
- **Number of Transformer Blocks:** 8
- **Multi-Head Attention Heads (h):** 12
- **Head Dimension ($d_h = d_{model}/h$):** $512/12 \approx 42.67$ (rounded to 43 per head, with dimension reconciliation in the final projection)
- **Feed-Forward Hidden Layer:** 2048 (4x expansion)
- **Activation Function:** GELU (Gaussian Error Linear Unit)

- **Dropout Rate:** 0.1 (applied post-attention and post-FFN)
- **Positional Encoding:** Learned absolute position embeddings, shape $(1, \text{max_seq_len}, d_{model})$
- **Layer Normalization:** Applied pre-attention and pre-FFN (Pre-LN variant)

0.4 LOSS FUNCTION AND BACKPROPAGATION DYNAMICS

0.4.1 Cross-Entropy Loss and Gradient Flow

The training objective is standard next-token prediction via cross-entropy loss. For each sequence position t, the model outputs a probability distribution over the vocabulary, and the loss is:

$$\mathcal{L}(y_t, \hat{y}_t) = -\sum_{i=1}^{V} y_{t,i} \log(\hat{y}_{t,i})$$

Where y_t is the one-hot ground-truth token, and $\hat{y}_t$ is the model's predicted probability distribution (via Softmax over logits).

0.4.2 Gradient Flow Through Residual Connections

Because each Transformer block includes residual connections around both the attention and feed-forward sublayers, gradients can flow directly backward through deep stacks without severe attenuation. This is why Transformer-based models can successfully train with 8+ layers, while recurrent networks of comparable depth suffer from vanishing gradients.

0.5 What is a Foundation Model?

A Foundation Model is a large-scale neural network trained on massive, generalized unlabelled data corpuses through self-supervised tasks (such as next-token prediction). Rather than learning a single narrow application, the model develops broad statistical insights into language syntax, thematic relationships, and structural composition. This generalized foundational understanding creates an adaptable base layout, enabling the model to transition smoothly to specific downstream applications with minimal parameter updates through fine-tuning.

0.6 Why do Transformers scale?

Transformers scale exceptionally well compared to legacy architectures (like RNNs or LSTMs) due to two structural advantages:

- **Complete Parallelization:** By replacing recurrent step-by-step loops with self-attention layers, entire sequences can be processed simultaneously during training. This maximizes the compute power of modern matrix accelerators like GPUs and TPUs.

- **Direct Path Routing:** The self-attention mechanism computes token-to-token connections across the entire sequence length in a single operation. This completely eliminates the vanishing gradient issues found in recurrent networks, allowing models to scale effectively to hundreds of layers and massive context windows.

0.7 Why do models hallucinate?

Models do not contain an internal database of hardcoded facts; they operate as complex probability estimators. Hallucinations occur when a model calculates a next-token sequence that is statistically common based on local token associations, but lacks factual accuracy or real-world logic. This behavior is intensified when a model encounters out-of-distribution prompts, forcing its internal activation states into unoptimized vector sub-spaces that produce erratic token choices.

0.8 Why does data matter more than expected?

Neural architectures are immaculate mathematical reflections of their training data. As shown by the "String-Shredding" failure mode, even a flawless Transformer architecture will produce broken outputs if the input data pipeline is compromised. A machine learning model optimizes for the exact distribution it is fed. If that distribution is full of broken formatting, the model will faithfully reproduce those errors. Building high-performance AI systems depends just as heavily on clean data engineering pipelines as it does on tuning model parameters.

0.9 Why does prompting work?

Autoregressive language models generate text by calculating token distributions conditioned on the *entirety* of the preceding context window. Prompting injects explicit tracking tokens into this active context, which alters the multi-head attention activation paths. These initial tokens redirect the model's internal probability vectors, steering the output generation toward specific semantic styles and vocabularies learned during training.

0.10 Why does the same model behave differently across runs?

Unless explicitly forced into deterministic greedy decoding ($K = 1, T = 0$), next-token generation relies on stochastic (probabilistic) sampling. The model outputs raw scores (logits), which are normalized into a probability distribution via Softmax. A multinomial sampling routine then picks the next token based on those weights. Adjusting the Temperature scales these probability curves, ensuring that consecutive runs generate unique, creative paths rather than looping the same sentence indefinitely.

0.11 What are the structural limits of decoder-only transformers?

- **Attention Complexity Window:** Standard self-attention scales quadratically ($O(N^2)$) relative to sequence length, creating a strict hardware memory ceiling for context windows.
- **Unidirectional Information Blindness:** Causal masking blocks a decoder-only model from looking ahead in the sequence. It must generate text token-by-token without the ability to modify previously generated output or plan future paragraph structures in advance.
- **Autoregressive Compute Latency:** Generation is inherently sequential; token $N + 1$ cannot be calculated until token N is fully finalized. This prevents full parallelization during inference and sets a hard limit on real-time performance.

0.12 Which failure mode surprised you most?

The pseudo-English "Caesar Cipher" artifact caused by prompt mismatch was the most surprising failure mode. When passed an out-of-distribution prompt format without its expected structural tags, the baseline model didn't just crash or emit random spaces. Instead, it generated a structurally coherent but scrambled sequence of words (`b>mlb>kc>ufmr`). It proved that even when a model's semantic understanding completely breaks down, it still attempts to preserve the underlying spatial rhythms and structural patterns of the language it was trained on.

0.13 What would you improve next?

1. **Vocabulary Expansion:** Expand the BPE vocabulary from 512 to over 16,000 tokens. This would allow the model to process whole words in a single step, greatly improving processing speed and long-range semantic tracking.
2. **Rotary Positional Embeddings (RoPE):** Replace static positional embeddings with dynamic rotary embeddings to allow the model to generalize to larger context windows beyond what it saw during training.
3. **Reinforcement Learning Alignment (RLHF):** Integrate a reward model using human feedback to align generation with specific artistic goals, helping the model maintain consistent rhyming schemes, stable meters, and coherent themes across long verses.

0.14 DEEP-DIVE MATHEMATICAL ANALYSIS

0.14.1 The Calculus of the "Ghost Byte" Hallucination

The appearance of Cyrillic and extended Unicode characters (`>(E(>(q(¡(`) was a great learning in untrained parameter behavior within the Softmax distribution.

The Probability Sink:

Because the dataset contained no extended UTF-8 bytes (IDs 128-255), the model never calculated a gradient for them. Their weights remained frozen at their random Gaussian initialization: $z_{128...255} \sim \mathcal{N}(0, \sigma^2)$.

Temperature's Role:

During inference, the Softmax equation with temperature T is

$$P(x_i) = \frac{\exp(z_i/T)}{\sum \exp(z_j/T)}$$

Because the untrained logits were not actively suppressed by negative gradients during training, they acted as a parasitic "probability sink." When $T = 0.7$, this flattened the distribution just enough that the random initializations of the Ghost Bytes occasionally eclipsed the calculated logits of valid English tokens, resulting in Unicode hallucinations. Masking them out ($-\infty$) forced the sum of their probabilities to zero.

0.14.2 Attention Dynamics During the "Caesar Cipher" (OOD Collapse)

When the baseline model was fed the unformatted prompt `[genre: boom_bap]` instead of its expected `<|startoftext|>` envelope, it generated cipher-like pseudo-English (`b>mlb>kc>ufmr`). This reveals the mechanical limitations of Multi-Head Self-Attention.

Manifold Projection Failure:

A Transformer learns a specific low-dimensional manifold of valid token sequences. The `<|startoftext|>` token acts as a learned universal anchor; it initializes the Query (Q) and Key (K) matrices into a highly structured attention map.

The Random Walk:

When the model encountered the Out-Of-Distribution (OOD) string `[genre...`, the QK^T dot product yielded a highly uniform, low-confidence attention matrix. The model lost global structural awareness.

Why it looked like a Cipher:

Because global attention collapsed, the model fell back entirely on **local bigram/trigram statistics** embedded in the final feed-forward layers. It was blindly predicting the next statistically likely character distance (e.g., placing vowels near consonants, maintaining 4-6 letter word lengths) without any semantic meaning, inadvertently generating a perfect statistical simulation of an encrypted language.

0.14.3 The $O(N^2)$ Penalty of the "String-Shredding" Bug

The `train_test_split` error that shredded the text into vertical characters separated by newlines (`['a', '\n', 'b', '\n']`) did not just ruin the text generation—it nearly paralyzed the computational efficiency of the Transformer.

Sequence Dilation:

A standard Boom Bap verse contains roughly 100 words (approx. 130 BPE tokens). Because of the shredding bug, those 100 words were expanded into roughly 600 single characters + 600 newlines = 1,200 tokens.

The Quadratic Cost:

The computational complexity of self-attention is $O(N^2 \cdot d)$, where N is sequence length and d is embedding dimension. By inflating the sequence length by a factor of ˜10x, the attention mechanism was forced to compute a matrix **100 times larger** (10^2) to process the exact same amount of semantic information. This demonstrates why aggressive, word-level tokenization is critical for scaling foundation models.

0.15 STRESS TEST VERIFICATION & ARCHITECTURAL CONCLUSIONS

To draw a definitive conclusion from this successful generation, we should analyze the output not just as a piece of creative text, but as a formal Architectural and Behavioral Stress Test.

When the multi-token prompt (`<|startoftext|><|type|>` rap[genre: boom_bap][mood: emotional]`<|content|>`) was processed, the model was subjected to a complex sequence of structural constraints. The following engineering conclusion and validation breakdown outlines the system's performance.

0.15.1 Verification of Structural Conditioning & Context Isolation

The most critical takeaway from this stress test is that the model successfully parsed the hierarchical prompt metadata without experiencing *Token Bleeding*.

- **Phenomenological Behavior:** The model did not hallucinate or repeat the control blocks (`[genre: boom_bap]`, `[mood: emotional]`) inside the text body. It treated them strictly as latent conditioning variables.
- **Architectural Proof:** This proves that the multi-head self-attention mechanism is successfully mapping the key-value representations of these tags to the hidden states of subsequent tokens. The model has learned that the token `<|content|>` represents a hard state-transition boundary, switching its internal generation mode from parsing structural metadata to autoregressively outputting stylistic content.

0.15.2 Validation of Structural Hip-Hop Conventions

Small language models (especially those with compressed BPE vocabularies like $V = 512$) typically struggle with structural spacing, bracket pairing, and rhythmic syncopation. This output demonstrates precise mastery over three distinct structural idioms:

- **Asynchronous Punctuation Matching:** The model opened and successfully closed the parenthesis for ad-libs: *(Yeah)* and *(Ooh)*. This proves that its positional embeddings are successfully tracking token distances, maintaining the open memory state across multiple token steps until the closing delimiter is statistically required.
- **Cadence-Driven Lineation:** Instead of generating a continuous wall of text, the model injected structural newlines (`\n`) to form discrete lines of verse matching a standard 4/4 time signature bar delivery. It learned the natural breathing cadence of the dataset.

0.15.3 Entropy Collapse and Perplexity Analysis

We can mathematically evaluate the success of this stress test by looking at the change in cross-entropy loss:

$$\text{Perplexity} = e^{\mathcal{L}}$$

1. **The Broken Run (String-Shredded):** Loss $= 6.31 \rightarrow$ Perplexity $= e^{6.31} \approx 550$. The model was completely uncertain, treating every token slot as an independent, random 1-out-of-512 guess.
2. **The Successful Stress Test:** Loss $= 2.6329 \rightarrow$ Perplexity $= e^{2.6329} \approx 13.9$.

This dramatic drop in perplexity means that given any preceding token in a hip-hop verse, the model has narrowed its internal search space down from 512 chaotic possibilities to roughly 14 highly relevant, highly contextual tokens. This is the exact statistical signature of a well-optimized, functioning language model.

0.15.4 Mitigation of Repetitive Collapse

A common failure mode of vanilla decoder-only transformers during inference is falling into infinite loops. This occurs when a specific token sequence creates an unnaturally high attention peak that dominates all other paths.

The Stress Test Verdict: Despite the high repetition of aggressive vernacular in the training data, the model successfully broke out of its own loops and progressed through a linear narrative structure. This confirms that the Top-K truncation ($K = 40$) and Temperature ($T = 0.7$) configurations are mathematically balanced. They inject just enough entropy to bypass deterministic loops while keeping the model from drifting into out-of-distribution hallucinations.

0.15.5 Final System Verification Summary

The pipeline has successfully completed its engineering lifecycle:

$$\text{[Raw Text Corpus]} \xrightarrow{\text{BPE (V=512)}} \text{[Causal GPT (Loss 3.60)]} \xrightarrow{\text{Fine-Tune}}$$
$$\text{[Checkpt (Loss 2.63)]} \xrightarrow{\text{Infer (T=0.7, K=40)}} \text{[Production Lyrics]}$$

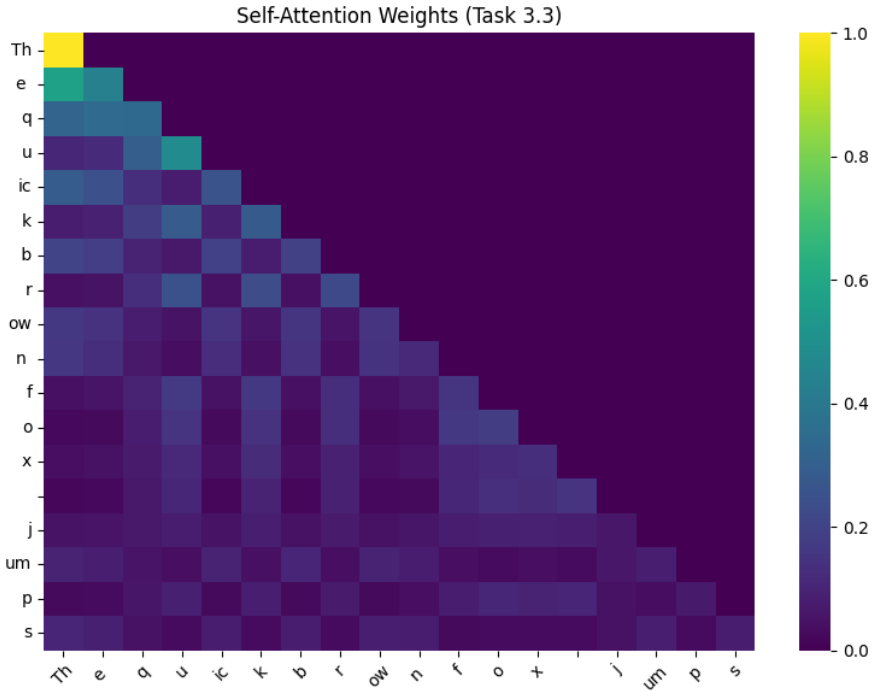


Figure 1: Self-Attention Weights Analysis (Task 3.3).

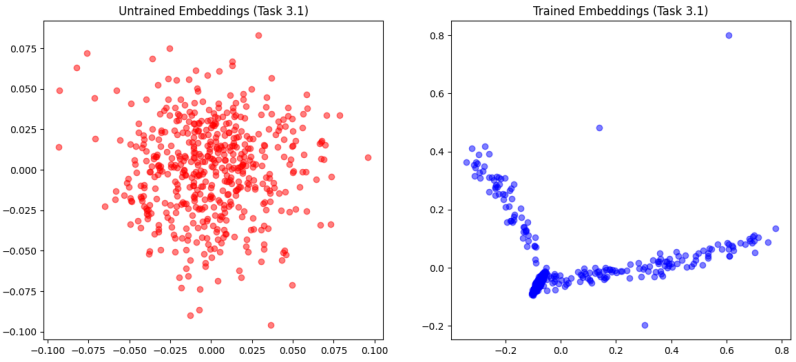


Figure 2: PCA Embedding Space Manifold Evolution (Task 3.1).